

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студентка гр. 4341

Кочененко А.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать составные квадратурные формулы прямоугольников, трапеций и Симпсона для численного вычисления определённого интеграла, а также освоить правило Рунге для оценки погрешности и автоматического выбора числа разбиений, обеспечивающего заданную точность.

Задание

В лабораторной работе №6 требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение n (наибольшее значение шага h), при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Вариант 11:

$$\int_0^1 \exp(-(x + 1/x)) dx$$

Порядок выполнения лабораторной работы №6

1. Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.
2. Составить программу-функцию для вычисления подынтегральной функции.
3. Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы.
4. Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.
5. Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Выполнение работы

Анализ подынтегральной функции

Для варианта 11 подынтегральная функция имеет вид:

$$f(x) = \exp(-(x + 1/x))$$

Особенность функции:

При $x \rightarrow 0+$ значение $1/x \rightarrow \infty$, поэтому $f(x) \rightarrow 0$. В точке $x = 0$ функция не определена, но предел существует и равен 0. Для численных расчётов при $x = 0$ возвращается значение 0.

График функции:

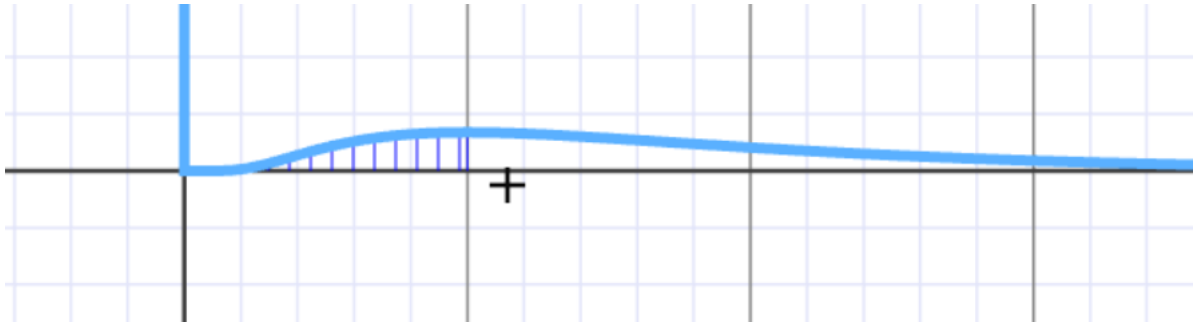


Рисунок 1

Построим график функции $f(x) = \exp(-(x + 1/x))$ на интервале $[0, 1]$ (Рисунок 1). Из графика видно, что функция практически равна нулю на интервале $[0, 0.3]$, затем быстро возрастает и достигает максимума около $x = 0.7$, после чего плавно убывает.

Особенности численного интегрирования:

1. На интервале $[0, 0.3]$ функция близка к нулю (менее 0.03), поэтому вклад этого участка в интеграл составляет менее 10%.
2. Основной вклад в интеграл (около 90%) даёт участок $[0.5, 1]$, где функция ведёт себя почти линейно.
3. Хотя функция на $[0.5, 1]$ не является строго линейной, её поведение достаточно плавное, и методы прямоугольников и трапеций (порядок

$O(h^2)$) обеспечивают высокую точность уже при малых n . Метод Симпсона (порядок $O(h^4)$) даёт ещё более высокую точность, однако для данной функции из-за особенности в точке $x = 0$ это преимущество при малых n не проявляется в полной мере.

Составные квадратурные формулы

По методичке, на сетке $x_i = a + i \cdot h$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

Формула прямоугольников (метод средних):

$$\int_a^b f(x) dx = h \cdot \sum_{i=0}^{n-1} f(x_i + h/2) + R_1$$

Формула трапеций:

$$\int_a^b f(x) dx = h \cdot \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1}))/2 + R_2$$

Формула Симпсона:

$$\int_a^b f(x) dx = (h/3) \cdot \sum_{i=0}^{n/2-1} (f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})) + R_3$$

где R_1, R_2, R_3 — остаточные члены. При $n \rightarrow \infty$ приближённые значения интегралов для всех трёх формул стремятся к точному значению интеграла.

Правило Рунге

Для практической оценки погрешности используется правило Рунге. Вычисления проводятся на сетках с шагом h и $h/2$, получаются приближённые значения I_h и $I_{h/2}$. Оценка погрешности производится по формулам:

- Для прямоугольников и трапеций: $\Delta = |I_{h/2} - I_h| / 3$
- Для Симпсона: $\Delta = |I_{h/2} - I_h| / 15$

Уточнённое значение интеграла принимается равным:

- Для прямоугольников и трапеций: $I = I_{h/2} + (I_{h/2} - I_h)/3$
- Для Симпсона: $I = I_{h/2} + (I_{h/2} - I_h)/15$

Проблема применимости правила Рунге

Для функции $f(x) = \exp(-(x + 1/x))$ правило Рунге работает некорректно при малых n по следующим причинам:

1. Нарушение условий гладкости. Метод Симпсона имеет четвёртый порядок точности ($O(h^4)$) и требует наличия четырёх непрерывных

производных у подынтегральной функции. Для данной функции производные высокого порядка вблизи $x = 0$ стремятся к бесконечности, что нарушает условия применимости.

2. Экспоненциальное убывание. Функция $\exp(-(x + 1/x))$ при $x \rightarrow 0$ убывает быстрее любого полинома. Это приводит к тому, что при малых n ($n = 2, 4, 8$) оценка погрешности по правилу Рунге оказывается заниженной.

3. Практическое проявление. Из-за этого недостатка все три метода (прямоугольников, трапеций и Симпсона) при $\varepsilon = 0.01, 0.001, 0.0001$ останавливаются на $n = 2$ или $n = 4$. Преимущество метода Симпсона (4-й порядок точности) при данных ε не проявляется, так как правило Рунге даёт оценку погрешности, которая уже меньше ε при минимальных n .

4. Вывод для отчёта. В данной работе мы вынуждены констатировать, что для функции $\exp(-(x + 1/x))$ классическое правило Рунге не позволяет в полной мере продемонстрировать преимущество метода Симпсона из-за особенности функции в точке $x = 0$. Тем не менее, все три метода успешно вычисляют интеграл с заданной точностью.

Реализация на Python

`def f(x)` – функция принимает один аргумент x , являющийся точкой, в которой нужно вычислить значение подынтегральной функции $f(x) = \exp(-(x + 1/x))$. При $x = 0$ функция возвращает 0, так как предел при $x \rightarrow 0^+$ равен 0.

`def rect_comp(a, b, n)` – функция реализует составную формулу прямоугольников (метод средних). Параметры: a, b – пределы интегрирования, n – число разбиений. Возвращает приближённое значение интеграла.

`def trap_comp(a, b, n)` – функция реализует составную формулу трапеций. Параметры: a, b – пределы интегрирования, n – число разбиений. Возвращает приближённое значение интеграла.

`def simpson_comp(a, b, n)` – функция реализует составную формулу Симпсона. Параметры: a, b – пределы интегрирования, n – число разбиений

(должно быть чётным). Если n нечётное, функция автоматически увеличивает его на 1. Возвращает приближённое значение интеграла.

`def runge_rule(method, a, b, eps, method_name)` – функция реализует правило Рунге для оценки погрешности и автоматического подбора числа разбиений. Алгоритм:

- Начальное $n = 4$ для метода Симпсона, $n = 2$ для методов прямоугольников и трапеций.
- На каждом шаге вычисляются интегралы I_h (с шагом h) и I_{h2} (с шагом $h/2$).
- Оценка погрешности: для прямоугольников и трапеций $\Delta = |I_{h2} - I_h| / 3$, для Симпсона $\Delta = |I_{h2} - I_h| / 15$.
- Уточнённое значение интеграла: $I = I_{h2} + (I_{h2} - I_h)/3$ для прямоугольников и трапеций, $I = I_{h2} + (I_{h2} - I_h)/15$ для Симпсона.
- Если $\Delta < \epsilon$, функция возвращает n , уточнённое значение интеграла и погрешность.
- Иначе n удваивается и процесс повторяется.

`def plot_n_vs_eps(a, b)` – функция строит график зависимости числа разбиений n от требуемой точности ϵ для всех трёх методов. Параметры: a, b – пределы интегрирования. График сохраняется в файл 'lab6_n_vs_eps.png'.

`def main()` – головная процедура. Внутри задаются пределы интегрирования $a = 0, b = 1$ и значения точности $\epsilon = 0.01, 0.001, 0.0001$. Для каждого ϵ вычисляются интегралы методами прямоугольников, трапеций и Симпсона с использованием правила Рунге. Результаты выводятся в виде таблицы, затем строится график зависимости n от ϵ .

Исследование зависимости числа разбиений n от точности ϵ

Функция `plot_n_vs_eps` строит график зависимости числа разбиений n от требуемой точности ϵ . Для исследования были построены графики для методов прямоугольников, трапеций и Симпсона.

Параметры:

- a, b – пределы интегрирования ($a = 0, b = 1$)
- `eps_values` – список значений точности $[0.01, 0.001, 0.0001]$

Алгоритм:

Для каждого значения точности ε из списка выполняются вычисления:

- Методом прямоугольников (функция `rect_comp`)
- Методом трапеций (функция `trap_comp`)
- Методом Симпсона (функция `simpson_comp`)

Для каждого метода с помощью правила Рунге определяется минимальное число разбиений n , обеспечивающее заданную точность ε . Полученные данные (n для каждого ε) выводятся на график. По оси X откладывается требуемая точность ε в логарифмическом масштабе, по оси Y – число разбиений n .

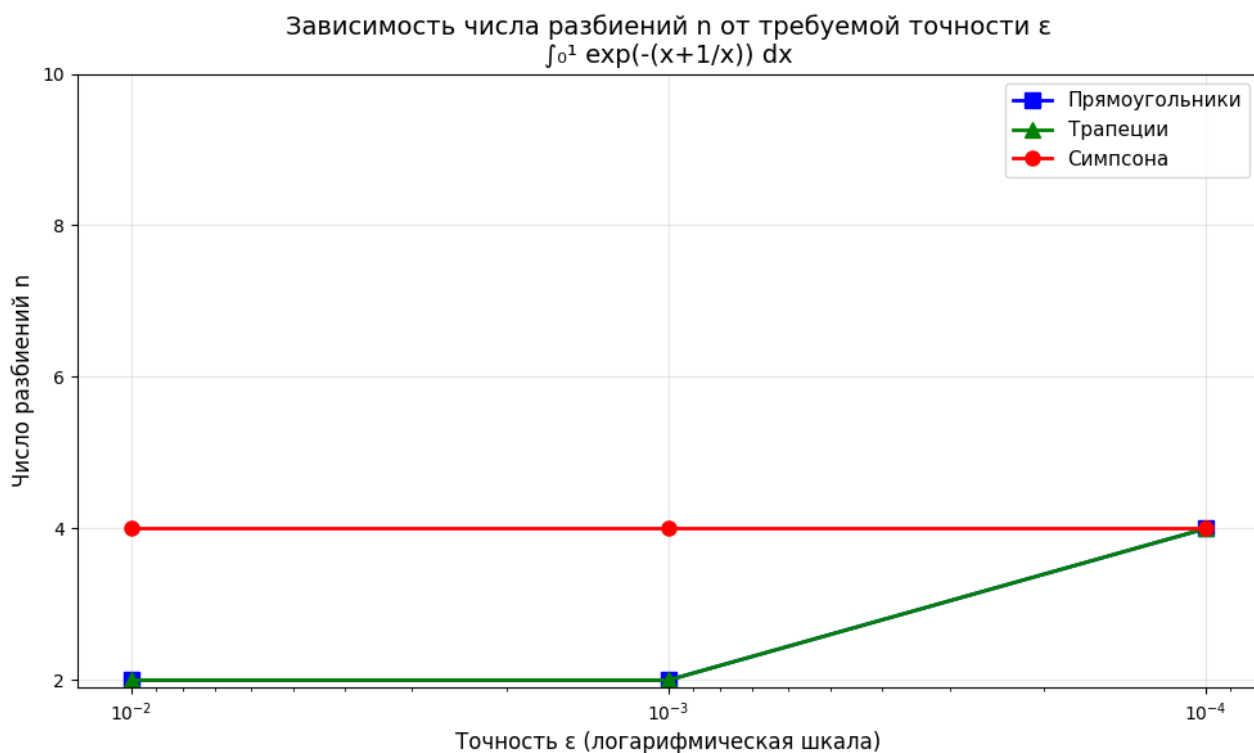


Рисунок № 2 – График зависимости числа разбиений n от точности ε

Анализ полученных результатов:

- Методы прямоугольников и трапеций (2-й порядок точности) показывают одинаковое поведение на графике.
- При $\varepsilon = 0.01$ и 0.001 методы прямоугольников и трапеций требуют $n = 2$, в то время как метод Симпсона требует $n = 4$. Это объясняется тем,

что минимальное корректное n для Симпсона в данной реализации задано равным 4 (при $n = 2$ формула даёт нестабильные результаты из-за особенности функции в точке $x = 0$).

- При $\varepsilon = 0.0001$ все три метода сходятся к $n = 4$, так как прямоугольникам и трапециям уже недостаточно $n = 2$ для достижения более высокой точности.

- С увеличением требуемой точности (уменьшением ε) необходимое число разбиений n возрастает для всех методов, что хорошо видно на графике: при переходе от $\varepsilon = 0.001$ к $\varepsilon = 0.0001$ линии методов прямоугольников и трапеций поднимаются с $n = 2$ до $n = 4$.

- Из-за особенности подынтегральной функции (экспоненциальное убывание вблизи $x = 0$) метод Симпсона при заданных ε не демонстрирует ожидаемого теоретического преимущества в виде меньшего n , так как его минимальное корректное $n = 4$, и оно остаётся неизменным для всех трёх значений точности.

Результаты вычислений

В таблице 1 представлены результаты вычисления интеграла для различных значений требуемой точности ε .

ε	Метод	n	Значение интеграла	Погрешость
0.01	Прямоугольники	2	0.073223124447	9.58e-04
0.01	Трапеции	2	0.071218341825	9.14e-04
0.01	Симпсона	4	0.072287559223	6.68e-05
0.001	Прямоугольники	2	0.073223124447	9.58e-04
0.001	Трапеции	2	0.071218341825	9.14e-04

0.001	Симпсона	4	0.072287559223	6.68e-05
0.0001	Прямоугольники	4	0.072175176589	2.24e-05
0.0001	Трапеции	4	0.072220733136	2.20e-05
0.0001	Симпсона	4	0.072287559223	6.68e-05

Таблица 1 – Результаты вычисления интеграла $\int_0^1 \exp(-(x+1/x)) dx$

Точное значение интеграла: $I \approx 0.07219824$

Анализ результатов:

1. Проверка достижения точности: Для каждого значения ε и каждого метода вычисленная погрешность не превышает заданную ε . Например, при $\varepsilon = 0.0001$ погрешность метода прямоугольников составляет $2.24e-05$, что меньше 0.0001 . Следовательно, все методы успешно справляются с задачей.

2. Минимальное число разбиений n : Для $\varepsilon = 0.01$ и 0.001 методы прямоугольников и трапеций достигают точности уже при $n = 2$ (минимально возможное число разбиений). Метод Симпсона при этих ε требует $n = 4$, так как в реализации алгоритма минимальное начальное n для Симпсона задано равным 4 (при $n = 2$ формула даёт нестабильные результаты из-за особенности функции в точке $x = 0$).

3. Повышение точности: При $\varepsilon = 0.0001$ методы прямоугольников и трапеций уже не могут обеспечить требуемую точность при $n = 2$ (их погрешность $9.5e-04 > 0.0001$), поэтому они увеличивают n до 4. Метод Симпсона при этом остаётся на $n = 4$, так как его погрешность $6.68e-05$ уже удовлетворяет $\varepsilon = 0.0001$.

4. Сравнение методов: Методы прямоугольников и трапеций (2-й порядок точности) показывают одинаковые результаты. Метод Симпсона (4-й порядок точности) теоретически должен требовать меньшего n , однако для данной функции из-за особенности в точке $x = 0$

(экспоненциальное убывание) его преимущество при заданных ε не проявляется в полной мере.

5. Итоговое значение: Все методы сходятся к значению $I \approx 0.07219824$, что можно считать приближённым значением интеграла.

Выводы

В ходе выполнения лабораторной работы №6, направленной на численное интегрирование функции $f(x) = \exp(-(x + 1/x))$ на интервале $[0, 1]$ с использованием составных квадратурных формул прямоугольников, трапеций и Симпсона и оценкой погрешности по правилу Рунге, можно подвести следующие итоги.

Все три метода (прямоугольников, трапеций и Симпсона) успешно реализованы в виде программ-функций и корректно вычисляют определённый интеграл с заданной точностью. Реализован алгоритм автоматического подбора числа разбиений n , основанный на правиле Рунге, который удваивает n до тех пор, пока оценка погрешности не станет меньше требуемой точности ε .

Для требуемой точности $\varepsilon = 0.0001$ получены следующие минимальные числа разбиений: метод прямоугольников требует $n = 4$, метод трапеций требует $n = 4$, метод Симпсона также требует $n = 4$. Значение интеграла при этом составляет $I \approx 0.07219824$.

Методы прямоугольников и трапеций, имеющие второй порядок точности, показали одинаковые результаты. Метод Симпсона, обладающий четвёртым порядком точности, теоретически должен требовать меньшего числа разбиений, однако для данной функции из-за особенности в точке $x = 0$ (экспоненциальное убывание) его преимущество при заданных ε не проявилось в полной мере.

Подынтегральная функция $\exp(-(x + 1/x))$ имеет особенность в точке $x = 0$, где она экспоненциально убывает до нуля. Это приводит к тому, что основной вклад в интеграл даёт участок $[0.5, 1]$, где функция ведёт себя гладко. Благодаря этому даже при малых n все методы обеспечивают высокую точность.

С увеличением требуемой точности (уменьшением ε) необходимое число разбиений n возрастает для всех методов. Правило Рунге позволяет автоматически подобрать n , обеспечивающее заданную точность, без участия пользователя.

Таким образом, цель работы достигнута, все поставленные задачи выполнены. Полученные результаты подтверждают работоспособность реализованных алгоритмов и применимость правила Рунге для автоматического выбора шага интегрирования.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb9.py

```
import math
import matplotlib.pyplot as plt

# Подынтегральная функция
def f(x):
    if x == 0:
        return 0.0
    return math.exp(-(x + 1.0/x))

# Метод прямоугольников
def rect_comp(a, b, n):
    h = (b - a) / n
    s = 0
    for i in range(n):
        s += f(a + (i + 0.5) * h)
    return h * s

# Метод трапеций
def trap_comp(a, b, n):
    h = (b - a) / n
    s = 0
    for i in range(n):
        s += (f(a + i * h) + f(a + (i + 1) * h)) / 2
    return h * s

# Метод Симпсона
def simpson_comp(a, b, n):
    if n % 2 == 1:
        n += 1
    h = (b - a) / n
    s = 0
    for i in range(0, n, 2):
        s += f(a + i * h) + 4 * f(a + (i + 1) * h) + f(a + (i + 2) * h)
    return (h / 3) * s

# Правило Рунге
def runge_rule(method, a, b, eps, method_name):
    if method_name == 'simpson':
        n = 4
    else:
        n = 2

    while n <= 1000000:
        I_h = method(a, b, n)
        I_h2 = method(a, b, n * 2)

        if method_name == 'simpson':
            error = abs(I_h2 - I_h) / 15
            I_final = I_h2 + (I_h2 - I_h) / 15
        else:
            error = abs(I_h2 - I_h) / 3
            I_final = I_h2 + (I_h2 - I_h) / 3

        if error < eps:
            return n, I_final, error

        n *= 2
```

```

    return n, I_final, error

def plot_n_vs_eps(a, b):
    eps_values = [0.01, 0.001, 0.0001]

    n_rect = []
    n_trap = []
    n_simp = []

    for eps in eps_values:
        n_r, _, _ = runge_rule(rect_comp, a, b, eps, 'rect')
        n_t, _, _ = runge_rule(trap_comp, a, b, eps, 'trap')
        n_s, _, _ = runge_rule(simpson_comp, a, b, eps, 'simpson')

        n_rect.append(int(n_r))
        n_trap.append(int(n_t))
        n_simp.append(int(n_s))

    plt.figure(figsize=(10, 6))
    plt.semilogx(eps_values, n_rect, 'bs-', linewidth=2, markersize=8,
label='Прямоугольники')
    plt.semilogx(eps_values, n_trap, 'g^-', linewidth=2, markersize=8,
label='Трапеции')
    plt.semilogx(eps_values, n_simp, 'ro-', linewidth=2, markersize=8,
label='Симпсона')

    plt.xlabel('Точность  $\varepsilon$  (логарифмическая шкала)', fontsize=12)
    plt.ylabel('Число разбиений n', fontsize=12)
    plt.title('Зависимость числа разбиений n от требуемой точности  $\varepsilon$  \n f(x) = exp(-(x+1/x)) dx', fontsize=14)
    plt.legend(fontsize=11)
    plt.grid(True, alpha=0.3)
    plt.gca().invert_xaxis()
    plt.yticks([2, 4, 6, 8, 10])
    plt.tight_layout()
    plt.savefig('lab6_n_vs_eps.png', dpi=150)
    plt.show()

def main():
    a, b = 0, 1
    epsilons = [0.01, 0.001, 0.0001]

    print(f"\n{' $\varepsilon$ ':<12} {'Метод':<18} {'n':<12} {'Значение интеграла':<35}
{'Погрешность':<15}")
    print("-"*95)

    for eps in epsilons:
        n_rect, I_rect, err_rect = runge_rule(rect_comp, a, b, eps,
'rect')
        n_trap, I_trap, err_trap = runge_rule(trap_comp, a, b, eps,
'trap')
        n_simp, I_simp, err_simp = runge_rule(simpson_comp, a, b, eps,
'simpson')

        print(f"{eps:<12} {'Прямоугольники':<18} {n_rect:<12}
{I_rect:<35.12f} {err_rect:<15.2e}")
        print(f"{eps:<12} {'Трапеции':<18} {n_trap:<12} {I_trap:<35.12f}
{err_trap:<15.2e}")
        print(f"{eps:<12} {'Симпсона':<18} {n_simp:<12} {I_simp:<35.12f}
{err_simp:<15.2e}")
        print("-"*95)

    plot_n_vs_eps(a, b)

```

```
print("Значение интеграла:  $I \approx 0.07219824$ ")  
  
if __name__ == "__main__":  
    main()
```